REPOSITORY REVIEW / SEPTEMBER 9, 2026

AsymptoticAnalysis

Incremental code audit and Mathics compatibility assessment

Native Wolfram comparison, new tooling findings, mathematical checks, and executable repair candidates

---

Repository https://github.com/VladimirReshetnikov/Asymptotic

Reviewed revision

513917b5b387152256b14ac76d92dd30cd13a11d

Novelty baseline 27 existing code-review packages; three waves.

Evidence Source inspection, independently executed Python checks, and explicitly attributed upstream records.

Abstract

This review evaluates the package as an asymptotic computation system and as a cross-interpreter software artifact. Its distinctive value is the combination of ordered real scales, retained branch information, structured remainders, and specialized inverse cores, not an absence of native Wolfram series or inversion facilities. The incremental findings concern code added after the most recent prior-review snapshot: interpreter-path normalization that escapes a POSIX virtual environment, incomplete timeout validation, exhaustive search inside a first-position adapter, and unbounded diagnostic capture. Two additional Mathics adapter questions are retained as unexecuted audit candidates rather than demonstrated mathematical failures. The accompanying code supplies tested Python corrections, a bounded-output prototype, exact independent mathematical checks, and focused interpreter characterization scripts.

Execution boundary. Twenty Python unit tests passed. No fresh Wolfram or Mathics execution of the package succeeded in this review environment. The article does not claim a new observed wrong public asymptotic formula or a complete compatibility certification.

Prepared for Vladimir Reshetnikov. Source references are commit-pinned; public documentation was consulted during this review. The archive contains editable LaTeX, the rendered PDF, code, and evidence, without checksum files.

Contents

1 Executive assessment

Overall judgment. AsymptoticAnalysis has a coherent mathematical purpose and a substantial implementation. It is more than a wrapper around `Series`: its real power–logarithm representation, branch-oriented inverse constructors, selected Lambert/Gamma/Barnes cores, and remainder-preserving operations address a useful research workflow. It should nevertheless be evaluated as a collection of admitted mathematical engines with different domains, not as an already complete transseries system or an already achieved replacement for all native asymptotic operations. The repository itself correctly distinguishes those ambitions from present coverage. [1, 2]

The most valuable incremental repairs found here are in the new Mathics support infrastructure. A correct symbolic algorithm is unavailable when its launcher selects the wrong Python environment. A per-case wall-clock limit is also not a complete resource contract when diagnostics accumulate without a byte limit. These are not criticisms of the asymptotic formulae; they are concrete weaknesses in the mechanism used to establish and reproduce compatibility.

ID	Priority	Incremental finding	Evidence here
N1	High	Resolving a virtual-environment interpreter symlink can launch base Python instead.	Actual Python mechanism reproduced.
N2	Medium	NaN, infinity, and extremely large finite timeout values pass validation.	Actual subprocess boundary failures reproduced.
N3	Medium	A first-position adapter enumerates all matches; a symbolic merge consumer can perform avoidable proof queries.	Source path plus independent operation counts.
N4	Medium	Captured kernel diagnostics have no byte ceiling.	Source resource analysis; bounded prototype tested.
A1	Audit only	The private limit adapter's default is one-sided, unlike modern native `Limit`.	Contract comparison; no public wrong-result witness.
A2	Audit only	The empty-map workaround does not cover analogous lookup helpers in the adapter context.	Source closure question; crash witness unrun.

Table 1: Priorities are this review's engineering assessment, not measured incident severity.

The recommendations are deliberately narrow. Preserve executable identity, validate the entire timeout domain, bound diagnostic capture without accepting truncated output as success, and make the new compatibility helpers earn their semantics through focused tests. Broad proposals already present in the earlier reviews—a universal result schema, general demand-driven refinement, complex sectors, and a general proof checker—are not repackaged here as new discoveries.

2 Scope, provenance, and novelty control

2.1 What was examined

The analysis is pinned to revision `513917b`. Inspection covered the repository README, kernel source map, the complete consolidated earlier-finding crosswalks, the Mathics compatibility guide, the new Mathics modules, the portable runner and its workflow, and selected production consumers in the main kernel, series operations, and Fourier coefficient code. The ordinary forward/inverse architecture was

examined through these interfaces and selected implementation boundaries. This is not a claim of line-by-line verification of every module, every vendored article, or every historical test record. [1, 2, 4, 5, 14]

The novelty screen used the maintained implementation register and wave-3 intake, which map all 167 attributed finding entries from the 27 reports. It also used the comparison against the last reviewed snapshot, `6687962`. The pinned revision is 29 commits ahead; the Mathics companion modules and the portable Mathics runner were added in that interval. This temporal distinction is especially useful: the concrete implementation mechanisms behind N1–N4 were not present in the last prior-review source snapshot. [3, 4, 5, 6]

That does not make every broad idea about testing or performance novel. N3 is a specific new exhaustive-search path, adjacent to the old general profiling item P08. N4 is a specific new subprocess resource boundary, adjacent to earlier validation concerns. Their value is the identified code path, failure or cost model, and repair acceptance criteria, rather than a new slogan about improving tests.

2.2 What was executed

The local environment provided Python 3.13.5 on Linux and a LaTeX toolchain. It did not provide an installed Mathics or Wolfram kernel. Attempts to reach the available remote Wolfram evaluator failed at the connection layer. Network access from the working container also did not provide a usable installation path. Consequently, the review distinguishes four evidence levels:

Observed mechanism. Real Python subprocesses established the virtual-environment switch and the exceptional timeout behavior. These tests exercise the same Python operations used by the runner, not the whole repository runner.

Tested candidate. Twenty Python unit tests exercise the proposed path/timeout fixes, patch transformation checks, and POSIX bounded-output prototype. The patch transformer was tested on source-fragment fixtures; it was not applied to and accepted by a complete downloaded checkout.

Independent mathematical check. Exact rational calculations check selected adapter formulae and operation-count models. They are independent oracles, not executions of Wolfram Language definitions.

Source-only or upstream evidence. Adapter risks are marked unexecuted. Existing native validation counts are attributed to their recorded upstream snapshots and are not promoted to fresh Mathics results.

The full package test suite was not run. The article's passing counts refer only to the named bundled Python tests and mathematical checks. No passing percentage for the package or for Mathics compatibility is inferred.

2.3 Earlier work intentionally excluded

The maintained register already covers native dense allocation, logarithmic remainder loss, ambient assumptions, equality of exact exponents, parameter capture in composition, refinement precision, native option/default handling, and generic observable Taylor admission. The current source includes substantial repairs to several of these. Repeating the old counterexamples as though they were current discoveries would be misleading. [4, 5]

For example, the existence of a native wrapper with a missing package analytic contract is not a newly found defect, nor is the known absence of a `DiscreteAsymptotic` backend. Those facts matter to the requested comparison, but are treated as coverage context. Similarly, the present report does not relabel the pending W3-06 Taylor-ingress concerns as new Mathics findings merely because the Mathics layer also calls a series provider.

A machine-readable novelty ledger in the archive records each new item, its closest older topic, and its evidence boundary. This is a mechanism-level novelty claim, not an assertion that all 27 full articles were independently reread in their entirety.

3 Mathematical and software architecture

3.1 The ordinary real power–logarithm contract

A useful conceptual model of an ordinary result is

$$f(x) = b(x) + P(x)\left[\sum_{\beta<h} w(x)^{\beta} C_{\beta}(\log w(x)) + \mathcal{O}\Big(w(x)^{\rho}(1+|\log w(x)|)^{d}\Big)\right], \qquad w(x) \longrightarrow 0^{+}. \tag{1}$$

Here b and P are retained exact components, the C_β are finite polynomials, and the source chart determines the positive small variable. The implementation uses sparse rows containing a weight and a logarithmic polynomial; it collects equal weights, normalizes coefficients, and transports precision separately from the displayed finite expression. Some specialized results have a different coefficient algebra or more than one carrier, so equation (1) is not a universal schema for every returned object. [7, 8]

The decisive mathematical distinction is between a finite expression and a reusable germ. For instance,

$$s(x) = x + \mathcal{O}(x^3), \qquad t(x) = -x + \mathcal{O}(x^3)$$

implies $s + t = \mathcal{O}(x^3)$, not exact zero. Erasing the remainder before algebra changes what is known. Conversely, retaining a remainder does not provide a numerical error bar: an unspecified big-O constant cannot certify an error at a particular point. This explains why the package separates finite normalization, asymptotic residual calculations, numerical comparisons, and rational interval certificates. [1, 7]

The positive chart is also mathematically substantive. At a finite endpoint, a left branch can use $w = x_0 - x$, while a right branch uses $w = x - x_0$. At negative infinity a positive reciprocal chart can use $w = -1/x$. Real noninteger powers are then admitted under branch hypotheses rather than treated as interchangeable complex principal powers. The current local-coordinate implementation makes these choices explicitly. [7]

3.2 Inverse engines and specialized scales

The ordinary inverse model normalizes a source into a translated leading monomial with higher-weight perturbations. Its coefficient calculus operates on complete weights rather than assuming one integer-degree ladder. Separate modules provide perturbative cores, logarithmic scales, flat exponential sectors, Fourier-polynomial coefficients, and specialized Gamma/Barnes inverses. The latter retain exact Lambert-type core expressions instead of needlessly expanding every slow variable into one long logarithmic formula. [2]

As a simple mathematical illustration, take $y = x + ax^{1+\delta}$ with $\delta > 0$, and seek the positive local inverse. Setting $x = yH$ and $t = y^{\delta}$ gives

$$H + atH^{1+\delta} = 1.$$

Writing $H = 1 + c_1 t + c_2 t^2 + c_3 t^3 + \cdots$ and equating coefficients yields

$$c_1 = -a, \qquad c_2 = (1+\delta)a^2, \qquad c_3 = -\frac{(1+\delta)(2+3\delta)}{2}a^3.$$

Thus the meaningful sequence of correction weights is $1 + n\delta$. Several independent irrational gaps create an additive weight set rather than a single rational lattice. This is the natural use case for an explicitly ordered sparse weight algebra. This derivation is an illustration of the representation, not a newly executed package test.

Specialized scales should not be mistaken for unconditional closure under all operations. The series-operation dispatch explicitly limits generic operations on Gamma/Barnes inverse kinds and redirects selected supported operations. A result may therefore be computable and refinable without admitting every generic observable or composition. That distinction is visible in the source and should inform application-level design. [8]

3.3 The Mathics layer is a semantic adapter, not an import shim

The new compatibility code does considerably more than define missing names. It supplies scoped early returns, association operations, held extraction, bounded exact assumption reasoning, local defining series, finite derivative sums, numerical-core spelling adjustments, refinement construction fixes, and presentation-specific rules. Its helpers are deliberately isolated from global `System` definitions. Some consumer definitions are rewritten after loading; other helpers are bound through a temporary adapter context while the source is parsed. [16, 17, 18, 19, 14]

This is a sensible design direction because a missing evaluator primitive can affect otherwise valid mathematics. A `Return` caught by the wrong loop changes control flow; a held association rule list changes metadata construction; a premature symbolic derivative index changes a finite coefficient formula. However, the adapter becomes part of the trusted execution path. Its scope must be narrow enough to audit and broad enough to cover the consumers it is intended to repair. N3 and A2 arise exactly at that boundary.

4 Comparison with native Wolfram Language

4.1 Compare the actual capabilities, not a Taylor-only caricature

Native `Series` already handles more than ordinary Taylor polynomials: its documentation includes negative and fractional powers, logarithms, expansion at infinity, and successive multivariate expansions. It also has assumptions and a policy for treating unrecognized functions as analytic. A comparison that describes it as an integer-power-only routine understates the native baseline. [27]

`SeriesData` has a rational exponent lattice in its direct coefficient representation, but native results can include logarithmic coefficients and symbolic factors outside that representation. Therefore, an irrational power appearing anywhere in an expression is not by itself evidence that native Wolfram Language cannot handle the expression. The stronger distinction is whether the desired algorithm orders, combines, and truncates a family of independent real weights while preserving the intended error contract. [28]

Native inversion is also real functionality, not an unimplemented comparison target. `InverseSeries` reverses admissible local series, including ramified examples. `AsymptoticSolve` attacks equations and systems with specified limiting behavior and can be applied to an inverse problem by writing $f(x) = y$.

Those native facilities and the package's branch-specific inverse constructors overlap, but they need not return the same representation or interpret the same numerical order request identically. [29, 30]

Table 2: A contract-oriented comparison. "Package" means the admitted analytic engines, not unrestricted native delegation.

Task	Native baseline	Package distinction / qualification
Local series	`Series`, `SeriesData`; ordinary and selected generalized local series.	Explicit real-weight rows and complete logarithmic blocks; positive source chart and separately retained remainder.
Inverse expansion	`InverseSeries` for series reversion; `AsymptoticSolve` for equations.	Dedicated inverse models, real branch admission, source observables, residuals, and specialized retained cores.
Broad asymptotics	`Asymptotic` for documented expressions and operators.	Selected analytic engines plus optional native routing; no completed universal coverage claim.
Discrete objects	`DiscreteAsymptotic` treats integer-domain asymptotics.	The required backend/coverage is not complete; continuous interpolation is not automatically an inverse of a sequence.
Reusable arithmetic	Native series participate in algebra and calculus.	Package objects retain additional domain/provenance/error data; generic closure depends on the result's scale.
Certification	Symbolic results and numerical computations are different forms of evidence.	A separate inverse certificate attempts rational enclosures under its own hypotheses; a displayed remainder is not that certificate.
Alternative runtime	Native names do not guarantee an implementation on another interpreter.	Mathics-specific exact adapters extend selected paths; native delegation is limited by the actual interpreter.

The native `Asymptotic` documentation describes an asymptotic mathematical contract, not merely arbitrary formal output. The package's decision to record `Missing["NativeContract"]` in its own remainder field means that it has not converted the native result into its internal analytic transport contract. It does *not* establish that all native asymptotic results lack mathematical guarantees. Keeping those two statements distinct avoids an unfair comparison. [31, 9]

The discrete comparison is similarly about domain semantics, not just a missing function name. An asymptotic formula along integer arguments can exploit properties absent on the real axis. An inverse of a discrete sequence requires a threshold, rounding, or interpolation convention before it becomes a real inverse-function problem. The package's declared complete-coverage goal includes discrete asymptotics, but the repository acknowledges that this backend and its full input coverage are not implemented. That is existing roadmap context, not a new defect credited to this review. [32, 1]

4.2 Match mathematical work before comparing cost

For an ordinary package power cutoff h, retained weights satisfy $\beta < h$. For an integer native `Series` order n, terms are requested through degree n. Thus an ordinary integer-power comparison often aligns $h = n + 1$, not $h = n$. In a factored expansion the cutoff may apply to the correction bracket rather

than the full product. A complete-block goal can also count one polynomial in reciprocal logarithms as a single unit. These are documented semantic choices. [1, 27]

A defensible benchmark should consequently define the mathematical target first: common chart, same branch, same retained terms, same required remainder frontier, and a stated coefficient algebra. Comparing identically written triples without that normalization can reward the implementation that computed less. A native expression retaining an exact core should not be charged with expanding that core merely to resemble a package pretty-printer, and the converse also applies.

For the new first-position path, the report therefore uses operation counts, not an invented runtime speedup. A reduction in predicate calls establishes a concrete optimization opportunity; it does not measure how much end-to-end inverse construction improves. Other normalization, sorting, simplification, and coefficient work may dominate.

4.3 Practical API and UX consequences

The existing explicit backend choice is useful when constructing reproducible experiments. A package-only request tests analytic admission, whereas an explicitly native request tests delegation and native preservation. Automatic routing combines those policies. The earlier reports already analyze defaults, option normalization, and routing surprises; this article does not repeat those findings. [5, 9]

For Mathics users there is an additional, previously separate layer of ambiguity: did the launcher actually select the intended environment? N1 means that a command can visibly name a virtual environment yet execute outside it. A resulting missing-module error may look like an installation problem, and a globally installed alternative version can make the error quieter. Reporting the chosen Python identity alongside interpreter output is therefore a direct consequence of this new finding, not merely another request for generic provenance metadata.

The package's own native and Mathics guides already explain loading order, positive chart conventions, finite normalization, and unsupported symbolic proofs. Those explanations should be retained. The actionable UX addition is an executable environment diagnostic and a precise refusal for invalid timeout values; users should not need to understand `Path.resolve` or C-level timeout conversion to diagnose either problem. [14, 10]

5 N1: interpreter normalization escapes a virtual environment

5.1 Source mechanism

In `validation/run_mathics_tests.py`, the Mathics branch chooses the requested Python executable, or defaults to `sys.executable`. When that value is an existing file, it is replaced by its resolved path before the subprocess is launched. The Wolfram executable branch uses a similar normalization. [10]

On POSIX, a virtual environment's `bin/python` may be a symlink to the base interpreter. The lexical executable path is not incidental: it lets Python locate the environment's configuration and select its prefix and site-packages. Resolving that path beforehand can therefore change the interpreter environment, rather than merely make the path easier to display. Python's virtual-environment documentation distinguishes the environment prefix from the base prefix and describes the environment's separate package installation context. [36]

5.2 Observed witness

The independent witness created a temporary virtual environment, added a uniquely named module only to that environment's site-packages, and launched the same small import twice. The first launch retained the virtual-environment path. The second applied the runner's resolving operation. No Mathics installation was needed to test the environment-selection mechanism.

Observation	Preserved executable path	Resolved executable path
Interpreter path	Temporary environment's `env/bin/python`	`/usr/bin/python3.13`
`sys.prefix`	Temporary environment directory	`/usr`
`sys.base_prefix`	`/usr`	`/usr`
Environment-only import	Exit code 0	Exit code 1; module not found

Table 3: Executed Python 3.13.5/Linux witness. Temporary directory names are preserved in the bundled raw evidence.

The result establishes a launcher defect under a common supported environment shape. It does not establish that every virtual environment uses symlinks or that every operating system behaves identically. A copied executable, Windows redirector, or managed distribution may have different mechanics. The regression should target both symlinked and copied environments rather than infer universal failure from one case.

5.3 Impact and why the current CI shape can miss it

The direct consequence is failed import of a dependency that is correctly installed in the requested environment. A second consequence is more subtle: an available global Mathics installation may be imported instead, so a run measures a different interpreter/dependency combination than the operator intended. A test report that records the transformed command will at least expose the selected path, but that does not make the original user request correct.

The workflow installs dependencies into the Python selected by setup-python and invokes the runner from that environment. It does not, by that structure alone, exercise an independently created symlinked virtual environment passed through `-python`. The documentation's recommended local virtual-environment workflow and the CI workflow therefore need a small explicit bridge test. [11, 14]

5.4 Repair

For an existing executable path, make it absolute without dereferencing its final symlink. Preserve a bare command name for normal PATH lookup. The bundled implementation is:

```
def interpreter_path(value):
    if not isinstance(value, str) or not value:
        raise ValueError("an executable name or path is required")
    if Path(value).is_file():
        return os.path.abspath(value)
    return value
```

This correction is intentionally narrower than a new launcher abstraction. It preserves the selected program's invocation identity and does not claim to install or discover Mathics. Both explicit and

default Python paths need the correction. The candidate also avoids dereferencing an explicitly supplied Wolfram launcher path; whether a particular Wolfram installation relies on such a path is a separate platform characterization, not a reproduced defect here.

5.5 Acceptance criteria

The decisive test is not string equality of two paths. It is that a fresh child imports an environment-only module and reports the intended prefix. Include a relative executable path containing spaces, a bare PATH command, a symlinked virtual environment, a copied-executable environment where available, and the default-executable route. The existing focused kernel suite should then be run through the selected environment, with the identity check performed before package evaluation.

The supplied Python unit tests cover path behavior and the actual POSIX environment-only import. The staged patch generator checks the inspected source fragments and refuses unexpected source shapes. It never edits the checkout in place. Its output is a candidate for integration, not a claim that repository acceptance has already passed.

6 N2: timeout validation does not define a safe input domain

6.1 Source mechanism and observed failures

The runner parses the timeout as a floating-point value and rejects nonpositive values. This leaves IEEE NaN outside the intended comparison: a test of whether NaN is nonpositive is false. Positive infinity and sufficiently large finite values also pass. They are then supplied to subprocess timing code. [10]

The independent witness exercised those accepted values against real Python subprocesses. On the local Python 3.13.5/Linux environment, the outcomes were:

Argument	Exception class	Observed diagnostic
`nan`	`ValueError`	Cannot convert float NaN to integer.
`inf`	`OverflowError`	Cannot convert float infinity to integer.
`1e100`	`OverflowError`	Timestamp too large to convert to C `PyTime_t`.

Exact exception text and timing implementation can vary with Python and the operating system. The portable conclusion is that the validator accepts values outside the intended usable deadline domain, not that every supported runtime must raise these exact exceptions.

The current subprocess exception cleanup is a positive feature: it attempts to terminate the owned child process before reraising unexpected exceptions. Nevertheless, these exceptions are not converted into ordinary complete case records by the runner's outer launch-error branch. They can interrupt the run and leave its last incremental report marked incomplete. This is *not* a false-success finding. It is a malformed-option and reporting failure that should be caught before any kernel starts. [10]

6.2 A complete narrow fix

Rejecting nonfinite values repairs NaN and infinity, but not the observed enormous finite value. The supplied candidate adopts a documented per-case range

$$0 < t \leq 86400 \text{ seconds}, \qquad t \text{ finite}.$$

The one-day maximum is a proposed CLI policy, chosen to be comfortably within ordinary subprocess timeout representations. It is not an existing repository requirement, nor a claim that every valid symbolic computation finishes within one day.

An alternative design can preserve arbitrarily large requested durations by implementing deadline logic with bounded wait slices. That is more complexity than this focused repair needs. An unlimited run should also be an explicit policy choice rather than an accidental consequence of passing infinity to a floating-point argument.

```
seconds = float(value)
if not math.isfinite(seconds) or not (0 < seconds <= 86400):
    raise argparse.ArgumentTypeError(
        "timeout must be finite and in (0, 86400] seconds")
```

Acceptance should include zero, negative values, NaN spellings accepted by the parser, infinity, an oversized finite exponent, an ordinary fractional timeout, and the documented upper endpoint. Invalid values must fail before process creation and produce an option diagnostic rather than a Python traceback. The bundled helper and tests exercise this policy.

7 N3: first-position lookup performs an exhaustive search

7.1 The new adapter and its production consumer

The Mathics first-position adapter calls `Position` with the requested level and head settings, obtains the complete position list, and only then selects its first element. There is no maximum-match argument or specialized early-stop path. The main kernel's symbolic `jetMerge` branch uses this helper to find an existing row with a provably equal weight; its predicate can invoke symbolic equality reasoning. [16, 7]

This is a concrete extra cost on a production consumer, not just a hypothetical list utility. It does not imply that the returned first position is wrong. Nor does this review rely on an undocumented guarantee about side effects inside native pattern predicates. The issue is the amount of traversal and proof work performed to answer a first-match query. Native documentation provides the relevant pattern/level/head contracts, but an optimized adapter still needs its own Mathics tests. [33, 34]

7.2 Cost model

Suppose a symbolic accumulator contains m already distinct weights. Append r rows whose weights equal the first accumulator weight, without canceling it. Once the initial accumulator exists, an exhaustive pass performs approximately rm predicate checks, whereas a first-hit search performs r checks on this suffix of the workload. This comparison excludes the initial seed construction and all other normalization costs.

The independent operation-count model gives the following values for $r = 100$:

Existing rows m	Exhaustive predicate checks	First-hit predicate checks
10	1,000	100
100	10,000	100
1,000	100,000	100
10,000	1,000,000	100

These are algorithmic counts from an independent model, not Mathics timings and not an asserted end-to-end speedup. In the actual merge consumer, row uniqueness can mean that there is only one returned match, so the primary waste is predicate traversal rather than a large list of matches. In a general recursive first-position call with many matching elements, the complete position list can additionally consume linear storage.

A duplicate-heavy input can make the distinction especially important because a failed symbolic equality proof is not necessarily cheap. Even when structural equality makes the first match trivial, the full search continues to the other rows. This new path is a sharper, localized opportunity than the earlier broad instruction to profile simplification.

7.3 Repair with a bounded semantic scope

The lowest-risk optimization is a specialized helper for the shape the symbolic merge actually uses: a flat list, level specification `{1}`, and `Heads -> False`. Scan list elements in order, return the first matching index, and evaluate the default only when no match exists. Retain the existing general fallback until its recursive level semantics are separately validated.

The archive contains `first_position_candidate.wl`, a non-mutating prototype with a distinct `Catch`/`Throw` tag. It is an *unexecuted* Wolfram Language candidate. Another possible implementation is a bounded `Position` call requesting at most one result, but support and behavior for that form must be checked on the target Mathics release before choosing it.

A focused acceptance run should record both the answer and the number of predicate evaluations for first, middle, last, and absent matches. It should check the laziness of the default, preserve the list/head/level contract, and include the actual symbolic merge consumer with complete coefficient comparison. A test that only verifies `FirstPosition[{a,2,b},_Integer]` finds position two cannot distinguish the exhaustive implementation from a stopping one.

8 N4: wall-clock bounds do not bound diagnostic memory

8.1 The resource gap

The portable runner merges standard error into standard output, captures the stream through `communicate`, and retains the resulting diagnostic string in the per-case report. There is no byte ceiling in that path. Python explicitly documents that this capture mode buffers data in memory and is unsuitable for unbounded output. [10, 37]

An OS-enforced time limit is valuable, but it does not bound how many diagnostic bytes a child emits before the limit. Moreover, the producer is not just a sequence of error messages: printing a very large exact result or expected expression can be expensive even when the mathematical computation has terminated. Accumulating those strings and repeatedly serializing reports creates a second resource demand in the parent process.

No real Mathics out-of-memory event was observed in this review. N4 is a source-established missing resource ceiling. It should not be inflated into a security exploit or a demonstrated production incident.

8.2 Do not make truncation look like success

A byte cap must be coupled to a failure policy. Silent truncation followed by the existing success parser would be unsafe: a stream could contain a nominal success marker and then exceed the allowed resource budget. Conversely, a truncated stream might omit the record's ending. The correct outcome is a distinct output-limit failure, with truncation explicitly recorded and no promotion to mathematical success.

The proposed state ordering is straightforward: launch failure, timeout, and output limit are terminal process outcomes; only an ordinary completed process is eligible for protocol parsing. A valid protocol record is still only evidence that the selected assertion completed. A raw zero exit status from an arbitrary characterization script is not a package acceptance claim.

8.3 Supplied prototype and its boundary

The POSIX-only `bounded_process.py` prototype drains output with a selector, stores at most a configured number of bytes, and terminates its owned process group when the byte cap or deadline is exceeded. Its live storage is bounded by the capture budget plus a fixed read chunk, rather than by the total output produced. The observed-byte field means bytes read before termination, not the unknowable total a killed child would eventually have emitted.

Five executed tests cover an ordinary result, nonzero exit, timeout, launch error, and an output-limit witness. In the last case a child attempts to emit 200,000 bytes with a 1,024-byte capture budget. The recorded outcome is `OutputLimit`, retained bytes are limited to the budget, and truncation is explicit. These tests are included in the total of twenty Python unit tests.

This is not a drop-in portable replacement for the repository runner. Windows needs its own tested process-tree or job-object treatment. Deliberately daemonized processes and arbitrary external work are outside the prototype's containment guarantee. The code is a reliability mechanism for owned test subprocesses, not a security sandbox. Integration must also preserve the repository's selected-case protocol and interrupted-run semantics.

9 Two Mathics audit candidates, not established public failures

9.1 A1: make the private limit default explicit

The Mathics limit wrapper declares an automatic default and maps it to numeric direction +1, meaning approach from below. Modern native `Limit` documents a two-sided real default. Explicit numeric and string directions have their own meanings; defaults for other functions, including asymptotic constructors, must not be substituted for the `Limit` contract. [20, 35]

The distinction is mathematically visible in

$$f(x) = \frac{|x|}{x}, \qquad \lim_{x \to 0^-} f(x) = -1, \quad \lim_{x \to 0^+} f(x) = 1.$$

There is no two-sided real limit. A private helper returning the left limit is not an implementation of the default two-sided request, although it can be correct for an intentionally left-sided internal query.

This review did not establish a public package path that omits the direction, reaches this wrapper, and consequently returns a false asymptotic theorem. A1 is therefore a latent adapter-contract mismatch requiring call-site characterization, not a claimed wrong `AsymptoticInverse` result.

The right repair depends on the consumers. If all relevant internal calls are intended to be sided, require or inject an explicit side at those call sites and document the helper as such. If default native equivalence is intended, compare both sides and prove agreement or retain a nonexistence/unresolved result. Simply changing the default mapping from below to above would not implement the native two-sided default either.

The bundled `limit-default` characterization case prints the adapter's result together with explicit left and right controls. Its output has not been collected in a real Mathics kernel here. It is a specification for the missing evidence.

9.2 A2: check workaround closure across helper contexts

The new list workaround is carefully scoped to package-private definitions. It avoids native Mathics mapping over an empty list because the repository documents an evaluator cache corruption that can later trigger an assertion when the list is reused inside a numeric list. The installer rewrites `System‘Map` occurrences in downvalues under the package's private context, excluding its own implementation. [21]

However, some analogous maps remain in the separate Mathics helper context. In particular, list-valued lookup helpers map recursively over requested keys or over associations. A query with an empty key list, or an empty list of associations, can therefore reach native mapping through a path outside the private-context rewrite sweep. The ordinary nonempty lookup tests do not establish the behavior of these empty cases. [16, 12]

The candidate witness is deliberately small: evaluate a lookup with an empty key list, assign its result, and then embed that result in a list such as `{result,2,0}`. A second case uses an empty list of associations. This is a helper-contract probe, not a claim that every public expansion constructs that particular metadata shape.

The proposed investigation should establish whether the cache-corruption preconditions actually survive this evaluation path. Object reuse, evaluator version, and the details of association application matter. Without executing the target release, source similarity alone is not enough to assert a crash.

If the witness reproduces, fix the corresponding helper cases explicitly or extend an audited list of consumers. Do not patch global `System‘Map`, and do not blindly rewrite every symbol in every context. The desirable invariant is that *each reachable empty-map use covered by the workaround's promise* receives the intended behavior. That is a more precise development obligation than either an unrestricted rewrite or a claim that namespace isolation alone proves compatibility.

10 Mathematical checks of the new exact adapters

This section records positive findings and independent checks. It helps separate limitations of the interpreter from defects in the mathematical formulae. The numerical counts below are model checks; the elementary proofs explain why the corresponding formulae are justified under their stated hypotheses.

10.1 Affine interval signs: strict interior behavior

The inverse-branch adapter uses endpoints of an affine expression on a deleted interval to prove sign conditions. The underlying argument is sound when its realness and positive-radius premises hold. [23]

Proposition 1. *Let $\rho > 0$ and let ℓ be a real affine function on $[0, \rho]$. Put $A = \ell(0)$ and $B = \ell(\rho)$. If $A, B \geq 0$ and at least one is positive, then $\ell(u) > 0$ for every $0 < u < \rho$. The corresponding nonpositive/negative statement follows by negation.*

Proof. For $t = u/\rho \in (0, 1)$, affinity gives

$$\ell(u) = (1 - t)A + tB.$$

Both weights are strictly positive. A nonnegative pair with at least one strictly positive entry therefore has a strictly positive convex combination. If both endpoints are zero, the affine function is identically zero. If the endpoints have opposite signs, this rule does not prove a common sign throughout the interval. □

The strictness point is important: requiring both endpoints to be strictly positive would unnecessarily reject a valid deleted-neighborhood proof. Conversely, testing just one positive interior sample would not prove the whole interval. The adapter's finite-real operand guards also matter; canceling an unproved complex offset from an ordered relation is not a substitute for establishing a real inequality. [23, 22]

The independent test model examined 966 asserted relation cases at 19 exact rational interior points each, for 18,354 checks, and retained 768 unresolved cases. There were no model failures. The finite samples do not prove the Wolfram Language implementation correct; they provide regression coverage beside the convex-combination proof.

10.2 Defining hypergeometric series and monomial transport

The defining-series adapter admits selected hypergeometric functions and polylogarithms at a vanishing monomial argument, with fixed parameters, bounded order, and a constant outer multiplier. For generalized hypergeometric functions it restricts to a nonzero convergence-radius regime and positive denominator parameters. These restrictions avoid claiming unrestricted analytic continuation from a local defining series. [24, 38]

For the hypergeometric part, the coefficient formula is

$$ {}_pF_q(\boldsymbol{a}; \boldsymbol{b}; z) = \sum_{k \geq 0} \frac{\prod_{i=1}^{p} (a_i)_k}{\prod_{j=1}^{q} (b_j)_k \, k!} z^k. \tag{2}$$

Positive lower parameters avoid denominator Pochhammer zeros. Fixed parameters are essential to the simple local statement: an error constant depending on them is not a uniform parameter bound. [38]

Proposition 2. *Suppose an analytic germ has a convergent series $F(z) = \sum_{k \geq 0} c_k z^k$ near zero. For fixed c and positive integer d, retaining all terms of $F(ct^d)$ through integer degree $N \geq 0$ leaves a remainder of order at least t^{N+1}.*

Proof. The first possibly omitted index is $k_0 = \lfloor N/d \rfloor + 1$. Analyticity bounds the remaining series by a constant times $|t|^{dk_0}$ in a sufficiently small neighborhood. Since $dk_0 > N$ and dk_0 is an integer, $dk_0 \geq N + 1$. Exact vanishing of further coefficients can improve the bound, but is not needed for this conclusion. □

The independent checker verifies 124 exact coefficient identities, using four families at indices zero through thirty:

$$
\begin{aligned}
{}_0F_1(;1;-x^2/4) &= \sum_{k\ge 0} \frac{(-1)^k x^{2k}}{4^k (k!)^2}, \\
{}_1F_1(1;2;x) &= \frac{e^x - 1}{x}, \\
{}_2F_1(1,1;2;x) &= -\frac{\log(1-x)}{x}, \\
{}_1F_1(-4;2;x) &= \sum_{k=0}^{4} \frac{(-1)^k \binom{4}{k}}{(k+1)!} x^k.
\end{aligned}
$$

The removable singularities in the middle two identities are interpreted by continuation at zero. An additional 124 integer degree checks verify monomial-order transport. These are exact coefficient and bookkeeping tests, not invocations of native `Series` or executions of the package adapter.

The restriction to a constant outer multiplier deserves preservation. If a surrounding factor contains a pole, a coefficient omitted from the inner local series can become relevant after multiplication. Extending that admission requires a corresponding order-demand calculation, not merely broadening a pattern. This observation concerns the rationale of the new guarded defining-series adapter; it does not claim to resolve the separate old generic Taylor-ingress findings.

10.3 Stirling order selection in the LogGamma adapter

The Mathics LogGamma path supplies a finite positive-real Stirling model when native large-argument series support is unavailable. For large positive z,

$$
\log\Gamma(z) = \left(z - \tfrac{1}{2}\right)\log z - z + \tfrac{1}{2}\log(2\pi) + \sum_{k=1}^{N} \frac{B_{2k}}{2k(2k-1)z^{2k-1}} + \mathcal{O}(z^{-(2N+1)}). \tag{3}
$$

This is a finite Poincare asymptotic statement, not a declaration that the full Bernoulli series converges. [25, 39]

Suppose the positive argument satisfies $z \sim au^{-r}$ with $a, r > 0$. The Bernoulli remainder in equation (3) is then $\mathcal{O}(u^{(2N+1)r})$. Given an exclusive correction cutoff h, the adapter selects

$$
N = \max\left\{0, \left\lceil \frac{h/r + 1}{2} \right\rceil - 1\right\}.
$$

For $N > 0$, the last retained Bernoulli power obeys $(2N-1)r < h$, while the first omitted power obeys $(2N+1)r \ge h$. When $N = 0$, the same omitted-tail inequality follows from the branch of the maximum that selects zero. Equality at the omitted power is consistent with an exclusive cutoff.

The independent checker tested 1,038 rational cutoff/rate pairs and found no bookkeeping failures. This supports the order-selection arithmetic. It does not prove that every argument is admitted correctly, every subsequent jet operation is sound, or the target interpreter executes the source as intended. The exact original argument is used to form the model before ordinary jet expansion, so replaying that source should not be mistaken, without further evidence, for discarding an unknown input tail. [25]

11 Mathics compatibility: what the evidence supports

11.1 A layered assessment

At the pinned revision, the project targets Mathics3 10.0.1 with scanner 10.0.1, SymPy 1.14.0, mpmath 1.3.0, and Python 3.11. These are the repository's specified test-environment versions, not the environment in which this review's independent Python checks ran. The guide explicitly says that complete compatibility is not yet established. [13, 14]

Table 4: Compatibility assessment without conflating levels of evidence.

Layer	Evidence in the inspected repository	Assessment in this review
Environment selection	Pinned requirements, recommended virtual environment, fresh-process runner.	N1 can invalidate intended interpreter selection; N2/N4 affect runner reliability.
Loading and isolation	Conditional bootstrap, modular/standalone entry points, removed adapter context, load/reload tests.	Substantial implementation; no fresh target-runtime load executed here.
Core language semantics	Scoped returns, lookup and held extraction helpers, assumption and list workarounds.	Positive design; N3 and A1/A2 require targeted follow-up evidence.
Ordinary symbolic calculus	Portable cases for forward/inverse work, algebra, refinement, and residuals.	Upstream focused examples, not a general proof of all public inputs.
Specialized mathematics	Defining series, finite Stirling model, selected exact core adjustments.	Independent formula checks support the guarded constructions; interpreter execution remains unverified here.
Numerical behavior	Selected quadratic/certificate and principal Lambert checks; nonprincipal numerical Lambert limitation documented.	Symbolic construction, numerical evaluation, and certification must be tested separately.
Native preservation	Historical Wolfram tests and later definition/isolation comparisons.	Evidence about the official-kernel path, not Mathics acceptance and not a new current full-suite result.

It would be inaccurate to say "Mathics is unsupported" based solely on its missing built-ins: the package implements a real compatibility layer. It would be equally inaccurate to say "fully Mathics-compatible" because selected loading and symbolic examples exist. The defensible assessment is **substantial, explicitly bounded compatibility work, with meaningful missing acceptance and a concrete launcher defect**. [14]

11.2 What the upstream preservation records do and do not mean

The compatibility guide records a historical official-Wolfram comparison with 1,452 passing tests and the same 12 existing failures across 79 suites, matching an untouched earlier baseline. It separately describes

later definition, context, builtin-isolation, and load/reload comparisons against updated controls. The guide expressly does not relabel the historical full run as acceptance of those later source changes. [14, 15]

That distinction is good evidence practice. Identical definitions on a selected official kernel are useful evidence that Mathics-only changes did not alter that kernel's path. They are not proof that every Mathics rewrite reproduces native behavior, and they do not settle version-dependent behavior on every surrounding user program. Nor do the unchanged historical failures disappear because preservation was successful.

The review did not independently rerun those records. The absence of a fresh runtime in this environment limits what can be concluded, but it does not erase the genuine upstream evidence. The correct response is attribution and scope, not either dismissal or unqualified endorsement.

11.3 Numerical and mathematical scope should remain separate

The principal ProductLog spelling adjustment illustrates why several compatibility dimensions matter. The package normalizes selected principal-branch constructions to a one-argument spelling to avoid a documented Mathics numerical argument-order problem. Its guide separately leaves numerical nonprincipal-branch behavior unsupported. An exact symbolic expression can therefore be useful even when numerical specialization of a related expression is not validated. [26, 14]

Likewise, the exact assumption helper proves selected realness and sign consequences from explicit conjunctions. It is not a replacement for quantifier elimination. Conservative refusal on an unsupported domain is preferable to manufacturing a branch, but it still does not count as successful compatibility for an input the project intends to support. The requirement is both truthful behavior and eventual coverage, not lowering the coverage target by calling every refusal a success. [22, 14]

12 Focused development recommendations

12.1 Repair the measurement path before expanding its claims

The first integration change should fix N1 and N2 together because they share the portable launcher boundary and have cheap, independent tests. Require an environment-only import smoke test on a symlinked POSIX virtual environment, then run a selected portable case through it. Record both the requested invocation path and the child's actual Python identity. Retaining these two values prevents a path-normalization mistake from hiding behind a plausible kernel-version string.

Add output-budget enforcement as a separate change. Its acceptance should use synthetic children, not expensive symbolic inputs, to exercise exact cap boundaries, success-marker-before-overflow, a timeout with partial output, and a child that exits nonzero after producing a complete-looking record. Only after these process outcomes are reliable should the wrapper feed an existing mathematical assertion parser. N4's prototype gives a tested POSIX starting point; a Windows implementation must earn its own evidence.

These are focused tests, not a proposal to rerun the full package suite indiscriminately. They target the infrastructure that selects and contains each future kernel case.

12.2 Test adapter consumers, not just replacement names

The existing primitive tests are useful, but a replacement can return the correct small example while behaving poorly on its actual consumer. N3 needs a duplicate-heavy symbolic merge test with predicate counts. A2 needs empty lookup results reused in the shape implicated by the workaround. A1 needs a caller audit that states whether each limit request is sided or genuinely default/two-sided.

A small compatibility-edge manifest can make this concrete. For each of the new helpers, record the admitted argument forms, the consuming production symbols, the relevant evaluator discrepancy, and the focused positive and negative tests. This is not a new universal result schema of the kind already proposed in D02. It is an implementation-local inventory for the newly added interpreter substitutions.

The manifest should distinguish syntax-time binding from late downvalue rewriting. A helper introduced before consumer parsing can capture names through a context path; a late rewrite can miss a different value category or context. A test asserting that no adapter context remains in the caller's search path establishes isolation, but it does not establish that every intended consumer was rewritten. Both properties deserve explicit tests.

12.3 Use exact small identities at new adapter boundaries

The independent affine, hypergeometric, and Stirling tests provide compact oracles that do not depend on agreement between two implementations sharing the same formula generator. Bring those oracles into target-runtime acceptance in a small number of carefully chosen cases: a strict deleted interval with a zero endpoint, a terminating hypergeometric polynomial, a monomial whose first omitted degree skips several powers, and an exact equality at a Stirling cutoff.

Each test should also contain a nearby unsupported case. For example, positive denominator parameters are not interchangeable with an arbitrary parameter hitting a Pochhammer pole; a nonconstant outer pole is not a constant multiplier; and unknown complex operands are not real affine inequalities. These controls check that broadening availability does not accidentally erase an admission hypothesis.

This recommendation is narrower than the earlier general differential-testing proposals. Its new contribution is the concrete adapter identities and boundary cases developed in this review, together with runnable independent oracles.

12.4 Avoid a large refactor as the first response

The current Mathics implementation is distributed across many small helpers and consumer rewrites. That can become difficult to maintain, but a large interpreter abstraction would be risky before the exact behavioral gaps are settled. N1 and N2 do not require one; N3 benefits from a small flat-list fast path; A1 requires a call-site decision; A2 requires evidence of the missing closure edge.

Once these cases are characterized, consolidate only demonstrably duplicated mechanisms. For example, a single audited empty-list mapping policy is useful if multiple contexts need the same workaround. A generic replacement that claims all `Map` semantics would be a much larger promise. Similarly, replacing all symbolic proof calls with a new global memoizer would be unrelated to the concrete first-match inefficiency and could introduce assumption-lifetime errors.

12.5 Prioritized acceptance plan

Table 5: New work packages and completion criteria.

Stage	Change	Completion evidence
1	Preserve interpreter path; validate timeout range.	Unit tests plus actual environment-only import through selected child; invalid timeouts rejected before launch.
2	Bound diagnostic bytes.	Explicit output-limit outcome; no success from truncated records; timeout and process cleanup controls on each supported OS.
3	Optimize the admitted first-position consumer.	Same merge coefficients and failures; reduced predicate counts on duplicate-heavy suffixes; no timing claims without measurements.
4	Characterize A1 and A2.	Real Mathics results for sided/default limit and empty-lookup reuse; public reachability classified separately.
5	Add exact adapter boundary cases.	Runtime results against the independent affine/Taylor/Stirling oracles, on modular and standalone entry points.

The broader native-input and vendored-asymptotic coverage obligations remain with the existing maintained plans. Completing this table would improve the credibility and reliability of Mathics validation, not close the entire project's coverage requirements. Existing items such as W3-06 or C08 should retain their original identities and acceptance obligations rather than be absorbed into a vaguely named compatibility refactor. [4, 5]

13 Artifacts, reproduction, and limitations

13.1 Archive contents

The archive contains this LaTeX source and its rendered PDF, Python fixes and tests, a source-shape-checked patch generator, a bounded POSIX subprocess prototype, independent mathematical checks, and unexecuted Wolfram/Mathics characterization scripts. The evidence directory records the executed Python results, environment details, and novelty/scope ledgers. It contains no checksum files and no bundled interpreter or font files.

Artifact	Status / purpose
`runner_fixes.py`	Tested path-preservation and timeout-policy helpers.
`apply_runner_patch.py`	Tested transformation mechanics; stages a candidate and diff, never modifies the checkout in place.
`bounded_process.py`	Tested POSIX resource prototype, not full portable-runner integration.
`run_independent_checks.py`	Executed launcher witnesses and exact mathematical/operation-count models.
`first_position_candidate.wl`	Unexecuted flat-list candidate; does not patch package definitions.
`runtime_contracts.wl`	Unexecuted interpreter characterizations and ordinary mathematical controls.
`run_runtime_probes.py`	Fresh-process characterization driver using preserved interpreter paths and bounded capture.

13.2 Reproduce the executed checks

From the extracted archive root, the standard-library-only tests can be run with:

```
python -m unittest discover -s code -p 'test_*.py' -v
python code/run_independent_checks.py \
  --output evidence/independent_checks_reproduced.json
```

The POSIX subprocess tests are platform-specific; the candidate does not claim Windows support. Check the script's command-line help when invoking on a different platform. A test rerun naturally changes temporary paths and timing details.

To stage the narrow launcher patch against a local checkout:

```
python code/apply_runner_patch.py /path/to/Asymptotic \
  --output /tmp/asymptotic-patched-runner.py
```

The generated diff is intended for review and application to a separate candidate checkout. Do not simply execute the staged runner from an arbitrary directory and assume it retains the repository-relative paths of the original file: the original runner locates its suite relative to its own location. After integration into a candidate checkout, execute the normal repository command there.

13.3 Collect the missing interpreter evidence

The characterization driver accepts a local package entry point and a specific interpreter path. For example, on POSIX:

```
python code/run_runtime_probes.py \
  --source /path/to/Asymptotic/src/Kernel/AsymptoticAnalysis.wl \
  --python /path/to/mathics-env/bin/python \
  --case first-position-count \
  --output evidence/mathics-characterization.json
```

Use a fresh process for each case. Repeat selected cases with the standalone entry point and with an official kernel where appropriate. The driver exposes `-wolfram` as the alternate interpreter selector. This review did not execute those commands successfully against either mathematical runtime.

A complete output frame means the characterization ran and printed its result; it is not a mathematical pass/fail assertion. In particular, A1 and A2 need interpretation of the actual output and, if a problem appears, a separate public-route test. The supplied driver is not a replacement for the repository's stronger source-snapshot and full assertion-report machinery.

13.4 Conclusion

The inspected code supports a favorable judgment of the package's mathematical organization and of the seriousness of its Mathics effort. The strongest new evidence is narrower than a universal correctness verdict: two reproducible Python boundary defects, one concrete adapter search-cost issue, and one missing diagnostic resource ceiling. The independent exact checks support selected newly added mathematical constructions rather than reveal a new asymptotic formula error.

The immediate development opportunity is therefore to make compatibility evidence more dependable before expanding its advertised scope. Preserve the requested environment, reject unusable deadlines, bound output, and test the exact adapter-to-consumer edges. Those changes are small enough to validate rigorously and specific enough not to duplicate the substantial earlier review program.

A Evidence counts and their meanings

Check family	Count	What the count establishes
Python unit-test methods	20 passed	Candidate helpers, patch fixtures, and bounded-process behavior.
Affine relation model	966 claims	Independent endpoint-rule claims checked at rational interior points.
Affine interior checks	18,354	Nineteen samples per asserted claim; not a substitute for the proof.
Affine unresolved cases	768	Model does not force a sign conclusion.
Exact coefficient identities	124	Four hypergeometric identities through index 30.
Monomial degree checks	124	First omitted degree meets the declared lower bound.
Stirling cutoff/rate pairs	1,038	Exclusive retained-order and omitted-tail arithmetic.
Wolfram package runs here	0 successful	Remote evaluation failed; no native acceptance claimed.
Mathics package runs here	0	Runtime not installed; characterizations remain unexecuted.

These populations are not added into one package test total. A formula identity, a subprocess unit test, and a complete package acceptance case are different forms of evidence. The raw JSON and test transcript preserve those distinctions.

Source conventions. Repository links below point to the reviewed commit except the explicit commit comparison. Source symbols and filenames are the primary locators; line numbers in the narrative identify the inspected runner region where useful but are not a substitute for the pinned revision. Public documentation links describe the relevant external contract as consulted during this review. Local evidence is in the accompanying archive and is not attributed to an external publisher.

References

[1] V. Reshetnikov, AsymptoticAnalysis repository README, reviewed revision.

[2] V. Reshetnikov, Modular kernel source map and Mathics compatibility map.

[3] V. Reshetnikov, Code-review report index: twenty-seven packages.

[4] V. Reshetnikov, Code-review implementation status and complete finding crosswalk.

[5] V. Reshetnikov, Wave 3: findings, proposals, and implementation scope.

[6] GitHub, Commit comparison from the wave-3 baseline to the reviewed revision; 29 commits ahead, including addition of the Mathics modules and runner.

[7] V. Reshetnikov, Core kernel source; public declarations, coordinate normalization, sparse jets, `jetMerge`, and forward series machinery.

[8] V. Reshetnikov, Explicit series operations; representation, capability guards, composition, and differentiation.

[9] V. Reshetnikov, Native result contracts, and native compatibility plan.

[10] V. Reshetnikov, Portable Mathics/Wolfram test runner; especially `run_case`, `main`, and command construction around lines 189–203.

[11] V. Reshetnikov, Mathics compatibility workflow.

[12] V. Reshetnikov, Portable exact regressions; loading and primitive cases.

[13] V. Reshetnikov, Pinned Mathics dependency requirements.

[14] V. Reshetnikov, Running AsymptoticAnalysis in Mathics3; supported examples, runtime subtleties, and validation boundaries.

[15] V. Reshetnikov, Historical Wolfram preservation and subsequent definition-comparison receipt; upstream evidence, not rerun here.

[16] V. Reshetnikov, MathicsCompatibility.wl; scoped returns, lookup, and first-position adapters.

[17] V. Reshetnikov, MathicsCalls.wl; held extraction.

[18] V. Reshetnikov, MathicsAlgebra.wl; bounded exact algebra.

[19] V. Reshetnikov, MathicsCalculus.wl; finite derivative sums and logarithmic Euler operation.

[20] V. Reshetnikov, MathicsSimplification.wl; simplification and limit-direction adapters.

[21] V. Reshetnikov, MathicsLists.wl; private empty-map workaround and installer scope.

[22] V. Reshetnikov, MathicsAssumptions.wl; bounded exact realness and sign reasoning.

[23] V. Reshetnikov, MathicsInverseBranches.wl; affine interval and polynomial branch helpers.

[24] V. Reshetnikov, MathicsTaylor.wl; guarded defining series at a vanishing monomial.

[25] V. Reshetnikov, MathicsSpecialFunctions.wl; finite LogGamma Stirling model.

[26] V. Reshetnikov, MathicsCoreFunctions.wl; principal ProductLog spelling and deadline preservation.

[27] Wolfram Research, *Series*, Wolfram Language documentation. `https://reference.wolfram.com/language/ref/Series.html`.

[28] Wolfram Research, *SeriesData*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/SeriesData.html.

[29] Wolfram Research, *InverseSeries*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/InverseSeries.html.

[30] Wolfram Research, *AsymptoticSolve*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/AsymptoticSolve.html.

[31] Wolfram Research, *Asymptotic*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Asymptotic.html.

[32] Wolfram Research, *DiscreteAsymptotic*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/DiscreteAsymptotic.html.

[33] Wolfram Research, *FirstPosition*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/FirstPosition.html.

[34] Wolfram Research, *Position*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Position.html.

[35] Wolfram Research, *Limit* and *Direction*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Limit.html; https://reference.wolfram.com/language/ref/Direction.html.

[36] Python Software Foundation, *venv—Creation of virtual environments*, Python documentation. https://docs.python.org/3/library/venv.html.

[37] Python Software Foundation, *subprocess—Subprocess management*, especially `Popen.communicate` and its memory warning. https://docs.python.org/3/library/subprocess.html.

[38] NIST Digital Library of Mathematical Functions, §16.2, *Definition and Analytic Properties of Generalized Hypergeometric Functions*. https://dlmf.nist.gov/16.2.

[39] NIST Digital Library of Mathematical Functions, §5.11, *Asymptotic Expansions of the Gamma Function*. https://dlmf.nist.gov/5.11.